



Jenkins

CI/CD

Automation for Google Compute Engine Deployment Using Jenkins Freestyle Job & Terraform



Jenkins



HashiCorp

Terraform



Google Cloud Platform



Jenkins.

Introduction

This project demonstrates how to automate the provisioning of a Google Cloud Compute Engine VM Instance using Terraform as IaC (Infrastructure as Code) and Jenkins Freestyle Job for CI/CD execution. Instead of a pipeline, the project uses a standard Jenkins job, making it simple yet powerful for production-grade workflow.

This automation ensures:

- ✓ No manual VM creation
- ✓ Repeatable, scalable infrastructure deployment
- ✓ Continuous delivery of cloud resources via Jenkins
- ✓ Full end-to-end DevOps Infrastructure Automation

Project Architecture

Developer → GitHub Repo → Jenkins Freestyle Job → Terraform Execution → GCP Provisioning

Flow Explanation:

- Developer writes Terraform template to create GCP VM
- Code is pushed to GitHub
- Jenkins pulls Terraform code using Git
- Jenkins executes terraform init → plan → apply
- New VM launches automatically on GCP

Prerequisites

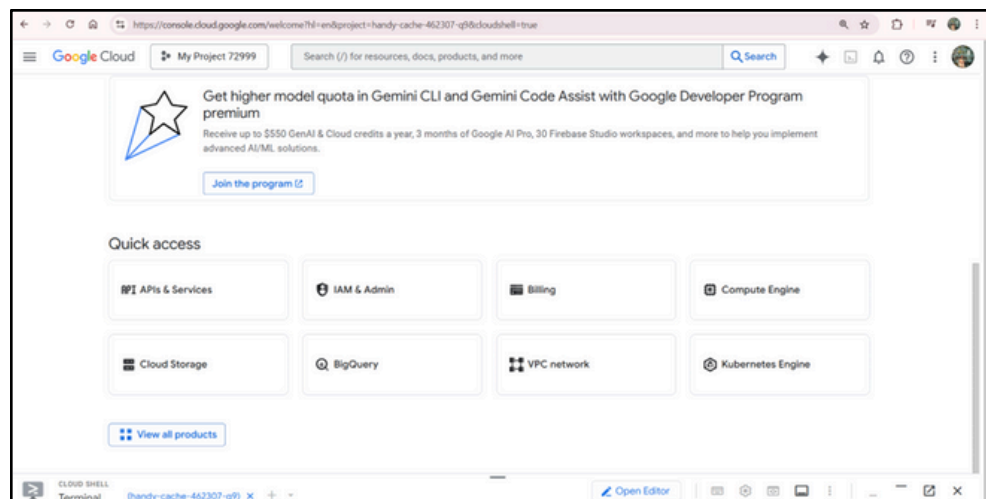
Requirement	Description
Ubuntu Server	Jenkins host
Jenkins Installed	Pre-configured with required
GCP Account	With billing enablement
Terraform Installed	On Jenkins machine
GCP IAM Service Account Key	For authentication



IMPLEMENTATION STEPS

Step-1: Create Service Account

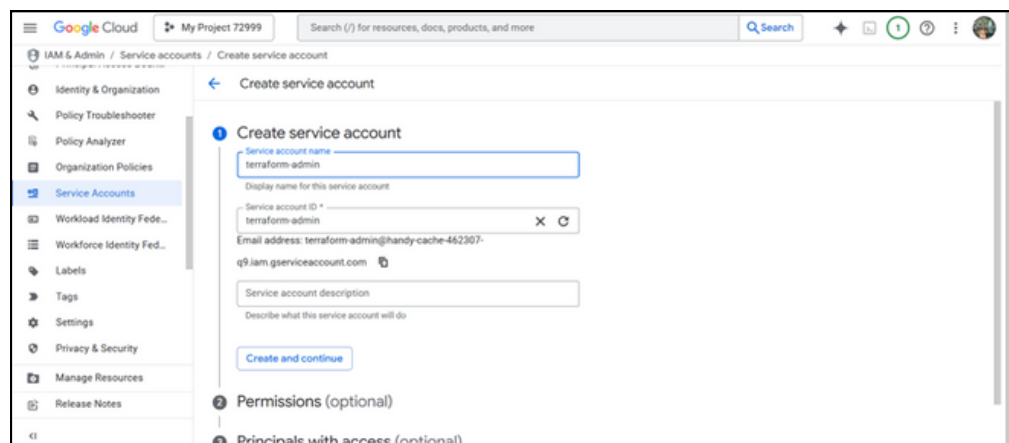
IAM & Admin



Service Accounts → Create Service Account

service account name = terraform-admin

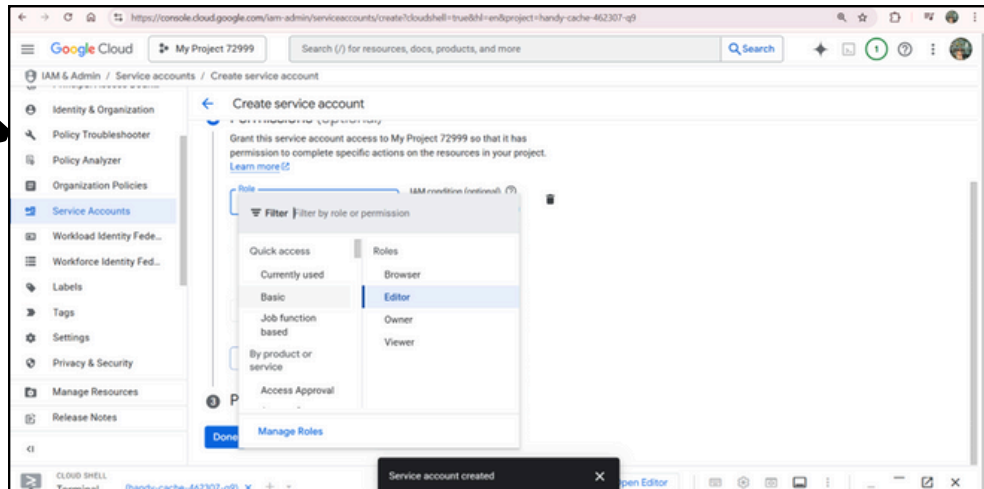
service account ID = terraform-admin



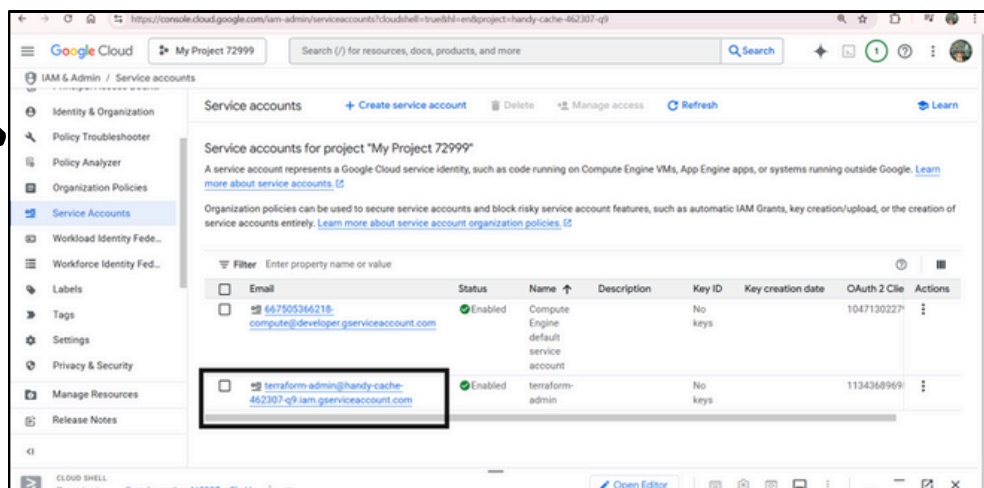


Jenkins.

Role: Editor

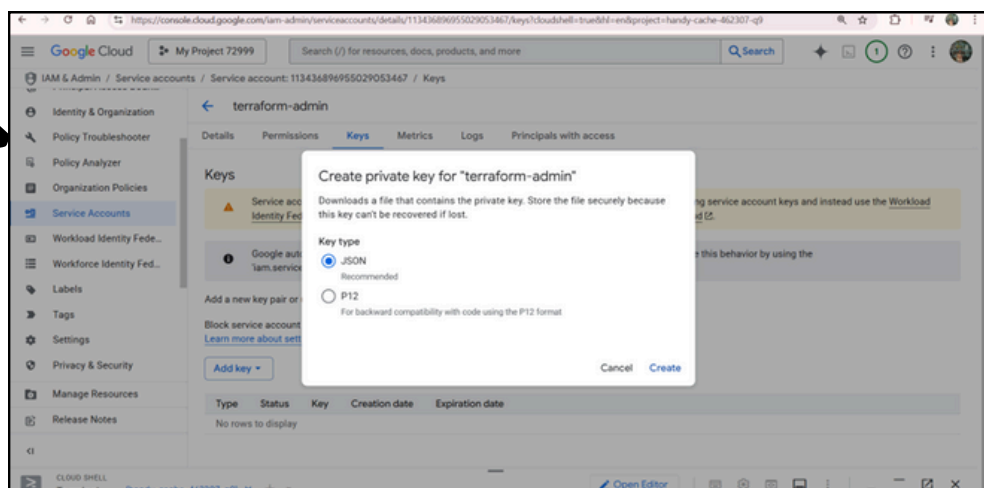


Check Service account is created



Step-2: Generate Key

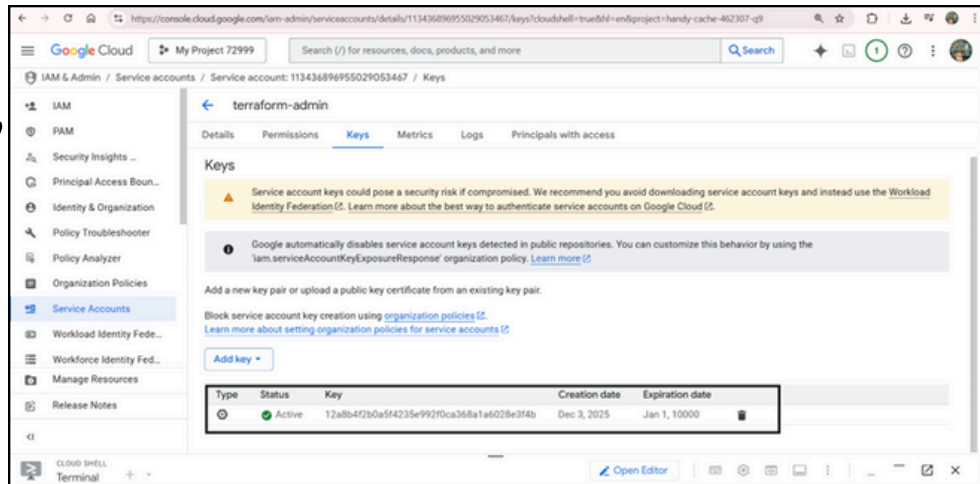
Download JSON key and save securely on Jenkins server:





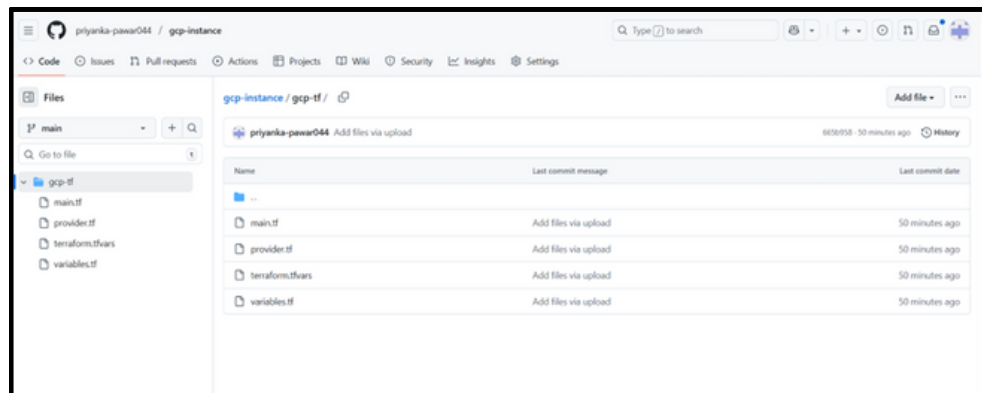
Jenkins.

Check Key



Step 3 : Create terraform GCP VM files

Terraform Code Structure



terraform files : <https://github.com/priyanka-pawar044/gcp-instance.git>

Step 4 : Jenkins Login

Launch an AWS Instance of Ubuntu

Install Jenkins on Ubuntu

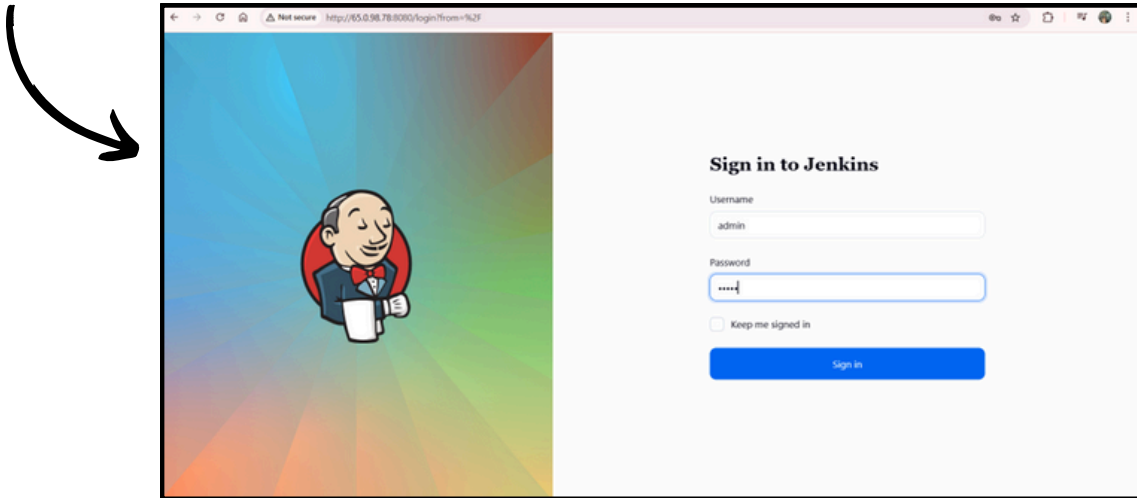
Install JAVA

Install Terraform



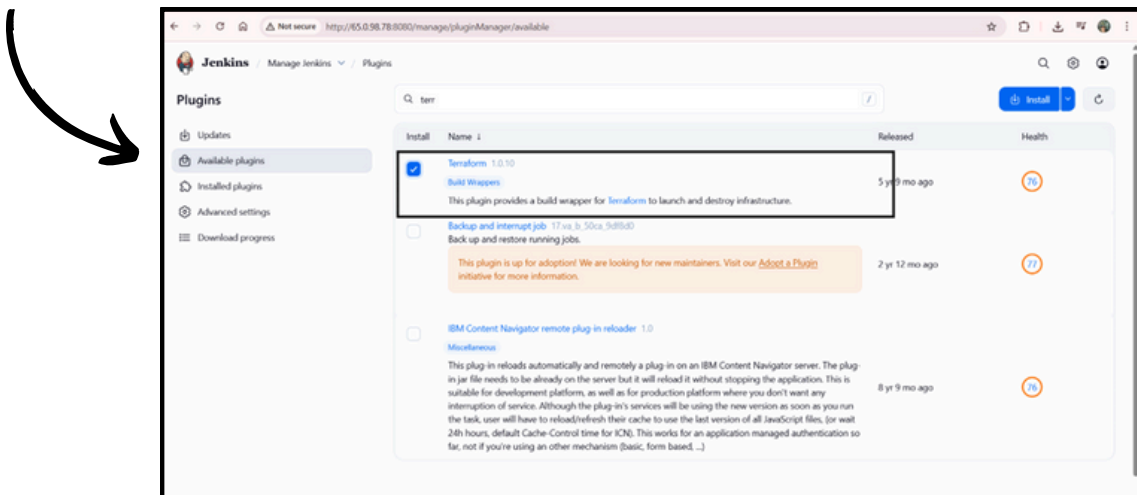
Jenkins.

Sign in to Jenkins



Step 5 : Jenkins Plugins Installation

Manage Jenkins → Plugins → Available Plugins



Step 6 : GCP Credentials Setup in Jenkins

To allow Terraform to authenticate with GCP, you must add Service Account JSON Key into Jenkins secure vault.

Manage Jenkins → Credentials → (global) → Add Credentials



Jenkins.

Field	Value
Kind	Secret File
Scope	Global
File	Upload gcp-service-account.json
ID	gcp-key (Important: we will use this

Not secure http://65.0.98.78:8080/manage/credentials/store/system/domain/_/newCredentials

Jenkins / Manage Jenkins / Credentials / System / Global credentials (unrestr...

New credentials

Kind: Secret file

Scope: Global (Jenkins, nodes, items, all child items, etc)

File: Choose File gcp.json

ID: gcp-key

Description:

Create

Click Create ✓

Check Credentials



Not secure http://65.0.98.78:8080/manage/credentials/store/system/domain/_/

Jenkins / Manage Jenkins / Credentials / System / Global credentials (unrestr...

Global credentials (unrestricted)

+ Add Credentials

Credentials that should be available irrespective of domain specification to requirements matching.

ID	Name	Kind	Description
gcp-key	gcp.json	Secret file	

Icons: S M L

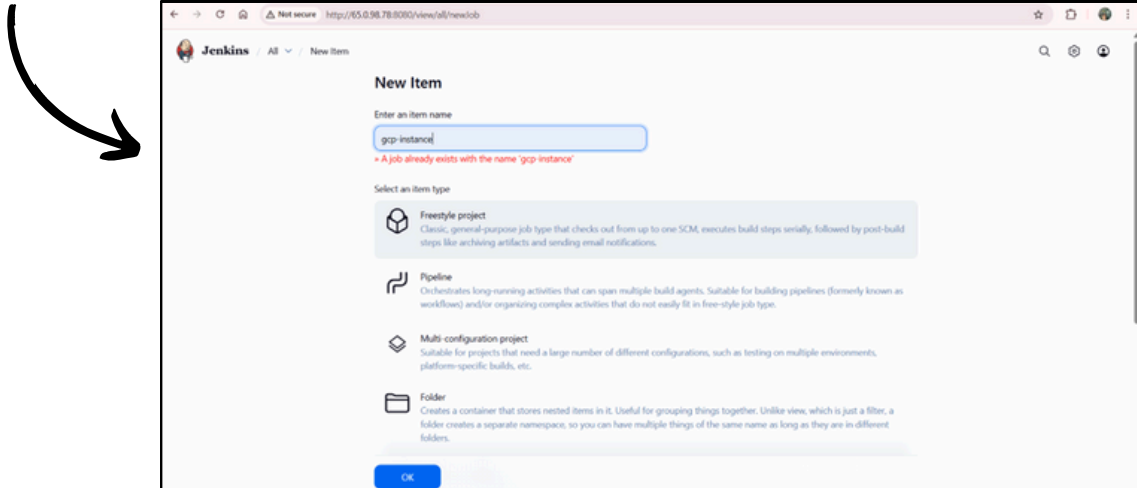


Jenkins

Step 7 : Create Freestyle Job

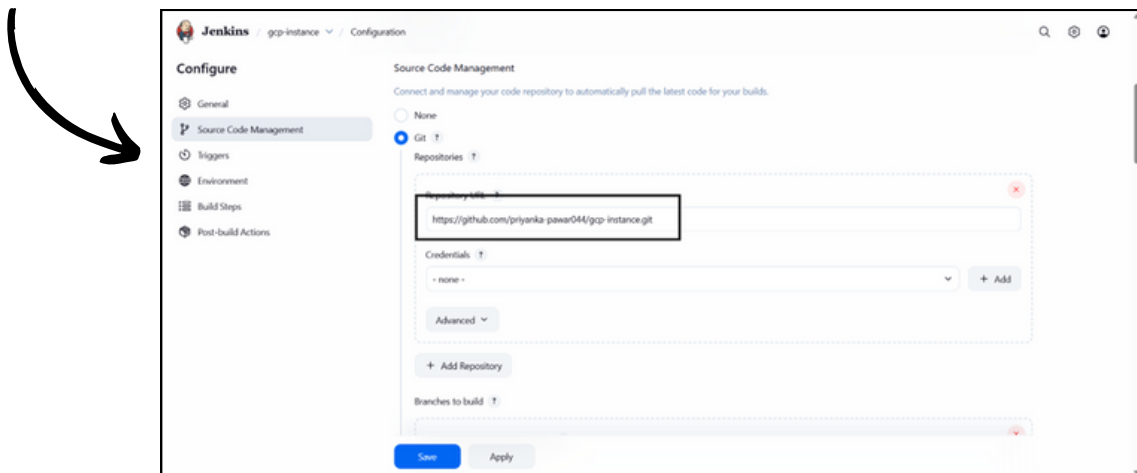
Dashboard → New Item → Freestyle Project

Name: gcp-instance

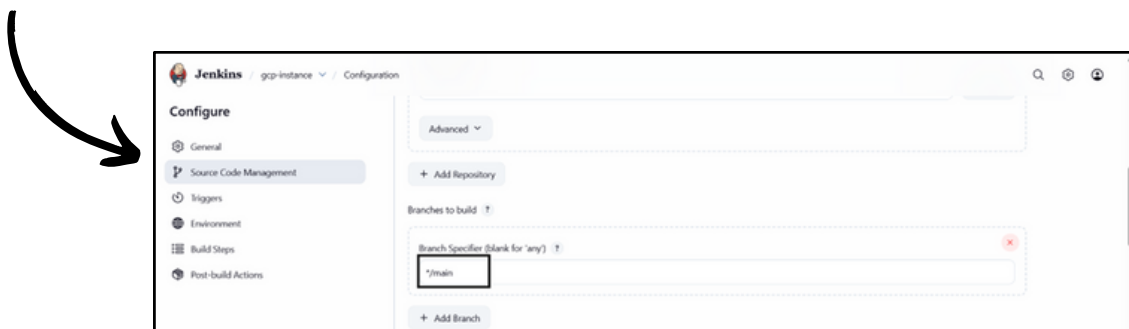


Source Code Management

Git Repository URL → <https://github.com/priyanka-pawar044/gcp-instance.git>



Branch: main/master

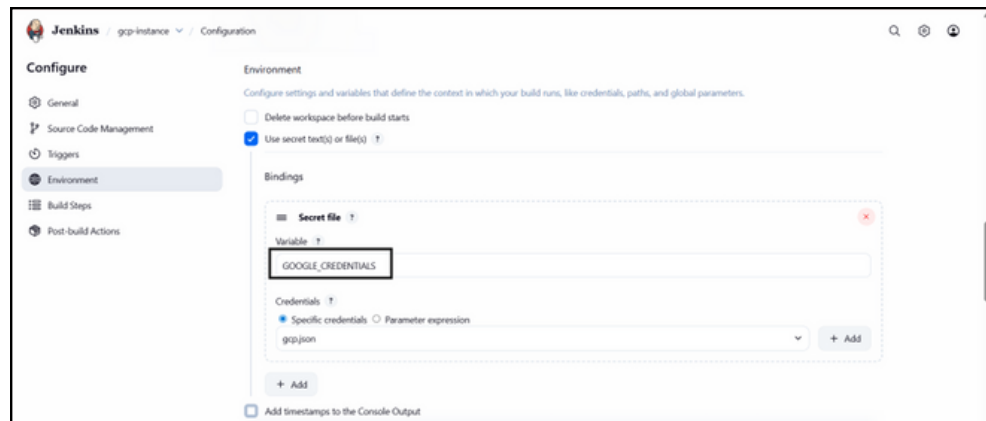
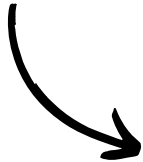




Jenkins.

Build Environment

- ✓ Enable → Use credentials
- Choose → gcp-key
- ✓ Enable → Inject environment variables



This will export auth key automatically before terraform runs.

Build Steps

Add **Execute Shell**

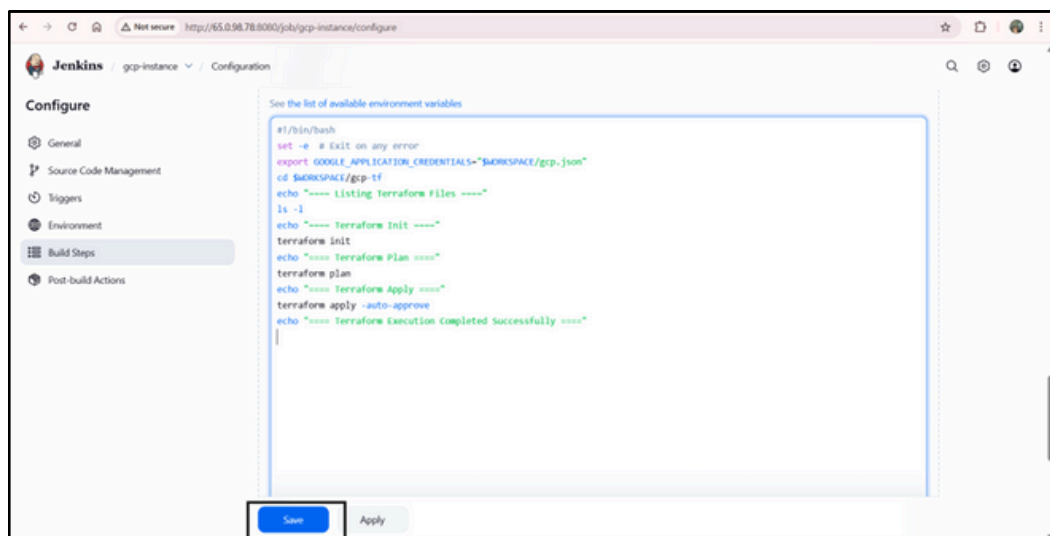
```
#!/bin/bash
set -e # Exit on any error
export GOOGLE_APPLICATION_CREDENTIALS="$WORKSPACE/gcp.json"
cd $WORKSPACE/gcp-tf
echo "==== Listing Terraform Files ===="
ls -l
echo "==== Terraform Init ===="
terraform init
echo "==== Terraform Plan ===="
terraform plan
echo "==== Terraform Apply ===="
terraform apply -auto-approve
echo "==== Terraform Execution Completed Successfully ===="
```



Jenkins.

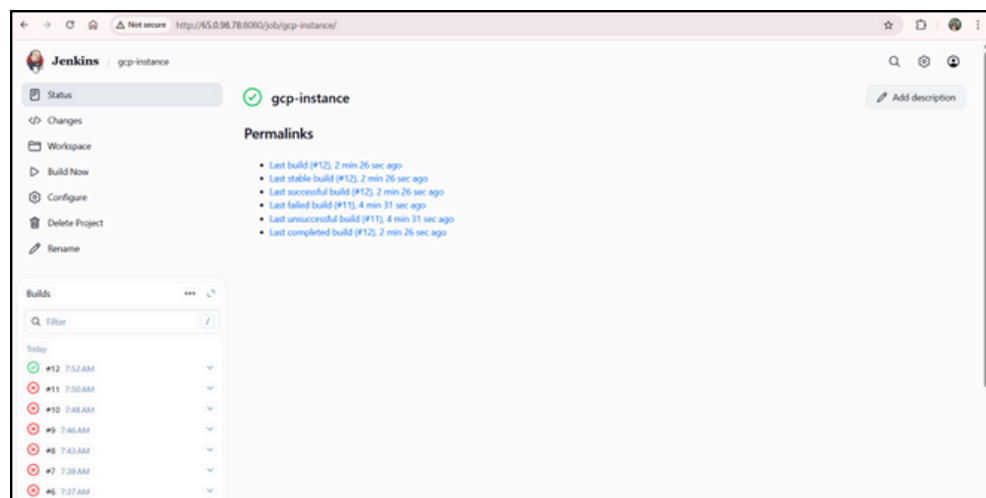
When this script runs:

- Jenkins identifies Terraform files
- Terraform initializes
- Terraform shows deployment plan
- Terraform automatically applies and deploys VM to GCP



Save and Click on Build Now

Terraform Execution Completed Successfully





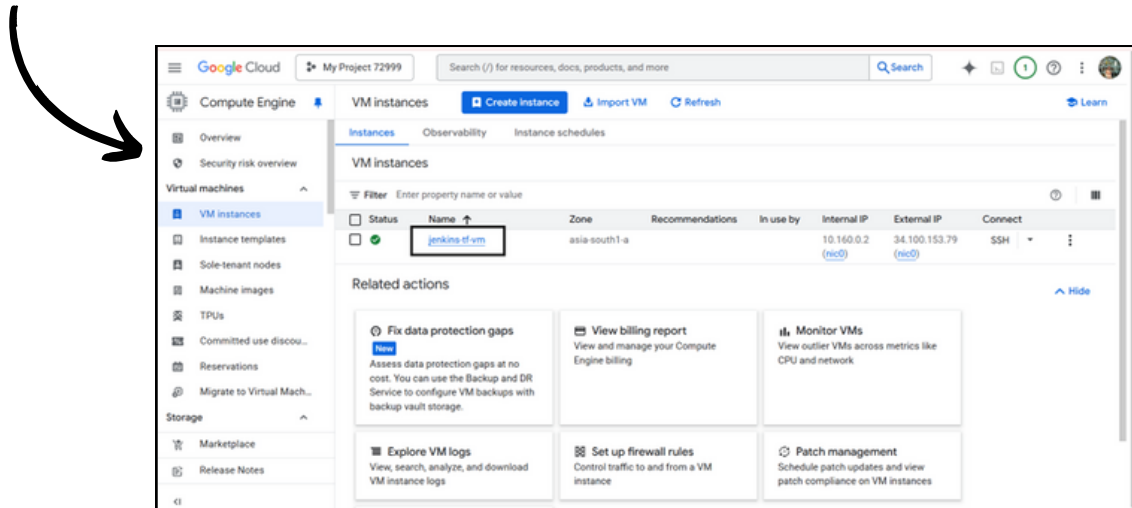
Jenkins

Step: 8 Verify on Google Cloud Console

Go to:

GCP Console → Compute Engine → VM Instances

If your Terraform ran successfully, you should see a new instance



If the instance is visible and RUNNING → SUCCESSFULLY LAUNCHED

Conclusion

In this project, we successfully automated the deployment of a Google Cloud Platform (GCP) virtual machine using Jenkins CI/CD integrated with Terraform Infrastructure as Code (IaC).

The entire provisioning process — from downloading Terraform configuration files to authenticating with GCP and launching the instance — was executed automatically using a Freestyle Jenkins Job.

Through this implementation, we achieved:

- ✓ End-to-end automated cloud instance provisioning
- ✓ Zero manual intervention with non-interactive terraform apply
- ✓ Secure authentication using service account credentials
- ✓ Repeatable, scalable, and production-style CI/CD workflow

This setup not only reduces deployment time but also removes human dependency and errors, offering an efficient, DevOps-ready infrastructure provisioning pipeline.



Jenkins.

We also implemented cleanup steps using terraform destroy, ensuring resource management and cost control.

This project demonstrates practical knowledge of :

- **CI/CD automation**
- **cloud provisioning**
- **Terraform-based IaC**

a valuable and real-time DevOps skill.

*Thank
You*